Controller Area Network
Mixed-Signal Modelling and Verification in eSim

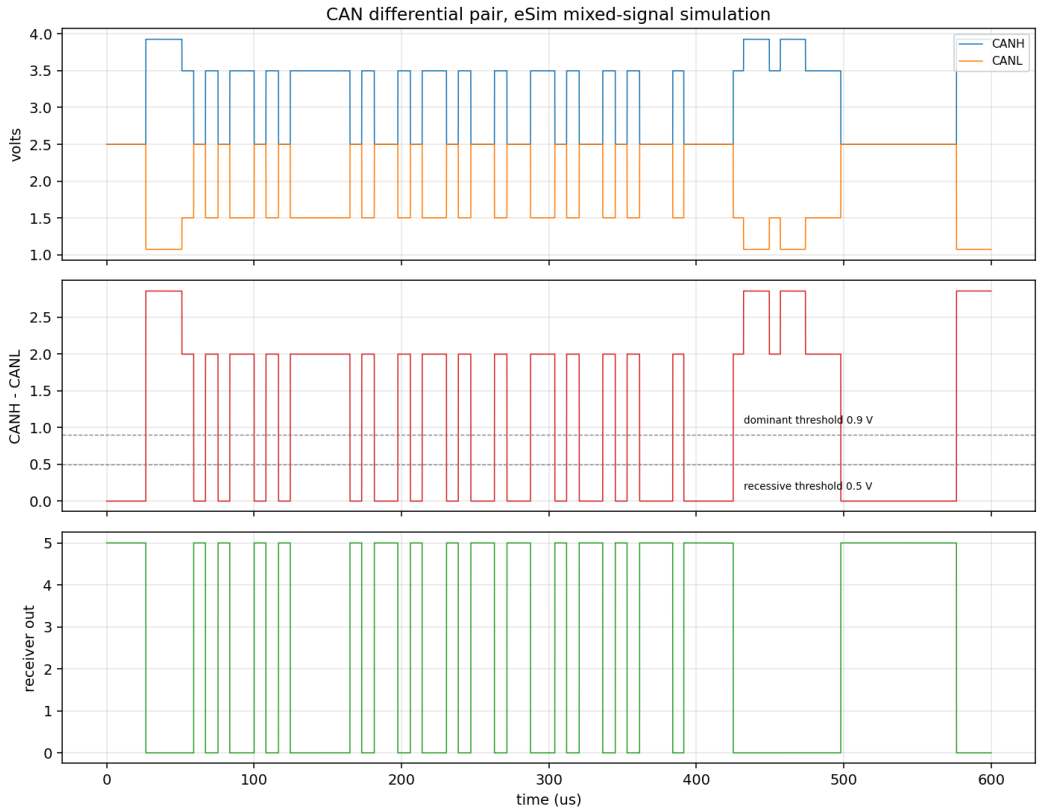
Complete Technical Report

Sourabh Kumar

Thapar Institute of Engineering and Technology, Patiala

FOSSEE eSim Semester Long Internship, Autumn 2026 — Screening Task 2

IIT Bombay



A complete CAN frame on the simulated differential bus. The elevated differential between 26 and 55 microseconds is two nodes arbitrating.

Contents

1. What this project is

This project is a complete Controller Area Network bus, modelled at circuit level and verified in simulation. It is not a behavioural model of a protocol, and it is not a digital design with analog decoration attached. It is a mixed-signal system in which the electrical behaviour of the wire and the logical behaviour of the protocol depend on one another, and neither can be removed without the other ceasing to work.

There are three pieces.

The physical layer is an Ngspice circuit: a differential pair with two driver stages, termination at both ends, a bias network, bus capacitance and a differential receiver. Real voltages, real impedances, real thresholds.

The protocol controller is ten VHDL modules implementing bit timing with resynchronisation, bit stuffing, a fifteen-bit CRC, frame assembly and decoding, arbitration, acknowledgement, error signalling and fault confinement.

The bridge is NGHDL, part of eSim, which compiles the VHDL with GHDL and wraps it as an XSPICE code model that Ngspice instantiates as a component. Analog-to-digital and digital-to-analog bridge elements convert between the continuous and discrete domains at every boundary.

Two controller instances share one differential pair. Asked to transmit at the same instant, they contend, and the node with the numerically lower identifier wins. The loser withdraws without corrupting the winner's message and immediately becomes a receiver of it. That behaviour is the heart of CAN and it cannot be demonstrated without a correct analog model, which is why this project exists in the form it does.

2. What CAN is, and why it exists

Controller Area Network was developed by Robert Bosch GmbH in the early 1980s and published as Version 2.0 in 1991. It is standardised as ISO 11898. It was created to solve a specific and expensive problem in automobiles.

2.1 The problem it solved

By the early 1980s cars contained a growing number of electronic control units: engine management, anti-lock braking, transmission control, climate control, instrument clusters. Each needed to exchange information with several others. Wiring every unit to every other point-to-point produces a harness whose cost, weight and failure rate grow with the square of the number of units. Twenty control units would need up to one hundred and ninety separate links.

The obvious answer is a shared bus, and the obvious problem with a shared bus is deciding who speaks. Ethernet's answer at the time was to let collisions happen, detect them, and have everyone back off for a random interval. That is unacceptable in a car. If the anti-lock braking system and the radio both want the bus, the braking message must win, and it must win deterministically, with a worst-case latency calculable in advance. A random backoff gives no such guarantee.

CAN's answer is arbitration that is both deterministic and non-destructive. Every message carries an identifier that also encodes its priority. When several nodes start transmitting simultaneously, the bus itself resolves the contest bit by bit, in real time, while the message is being sent. The highest-priority message proceeds without interruption and without losing a single bit. The losers withdraw silently. No bandwidth is wasted on the collision, and the latency of the highest-priority message is unaffected by how many other nodes wanted to transmit.

2.2 How the bus makes that possible

The mechanism depends entirely on an electrical property. CAN defines two bus states:

State	Logic	Electrical behaviour	Differential
Recessive	1	passive; bus left to its bias network	0 V nominal
Dominant	0	actively driven by a low-impedance driver	2 V nominal

A node driving dominant physically overrides any number of nodes driving recessive, because a low-impedance driver wins against a passive bias network. The bus behaves as a logical AND of every node's output: recessive only if every node is recessive.

This is called a **wired-AND**. It is not a metaphor and it is not implemented with a gate. It is a consequence of the driver topology and the resistor network, and in this project it is modelled as such — two driver stages of voltage-controlled switches sharing one terminated differential pair.

2.3 Arbitration as a physical process

Every transmitting node monitors the bus while transmitting. The rule is one sentence:

> A node that transmits a recessive bit and reads back a dominant bit has lost arbitration. It stops transmitting immediately and becomes a receiver.

The converse cannot occur: a node transmitting dominant always reads dominant, because its own driver holds the bus there. The test is one-sided, and the loser is always the node with a recessive bit where another has a dominant bit — the node with the numerically higher identifier at the first differing bit position.

Because identifiers are sent most-significant bit first, and dominant is logic zero, the numerically lowest identifier wins. Priority is encoded directly in the identifier, and arbitration is resolved by the wire rather than by protocol logic.

2.4 What else CAN provides

Synchronisation without a clock line. There is no clock wire. Each node has its own oscillator and recovers timing from transitions on the data line.

Bit stuffing to guarantee transitions occur often enough for that resynchronisation to work.

A fifteen-bit CRC with Hamming distance six for frames up to 127 bits, so up to five randomly distributed bit errors are always detected.

An acknowledgement slot in which every node that received the frame correctly asserts a dominant bit.

Five independent error detection mechanisms and a fault-confinement scheme that removes a persistently faulty node before it disrupts the network.

3. Why CAN was chosen

The task permits any protocol and suggests UART, SPI or I²C. CAN was chosen instead, deliberately.

3.1 The simpler protocols do not need the analog domain

UART is two shift registers and a baud-rate divider; its behaviour is entirely digital and the analog layer, if present, has no influence on whether the protocol works. SPI adds a clock and chip selects but remains digital in its logic. I²C is closer — open-drain with pull-ups and genuine wired-AND behaviour — but its arbitration is rarely exercised and its analog aspects reduce to an RC time constant.

A model of any of these in eSim would be a digital design with analog components attached for appearance. The task asks for circuit-level modelling and verification; a protocol whose correctness is independent of the circuit does not demonstrate that.

3.2 CAN cannot be modelled without the circuit

Arbitration requires a node to read back the actual voltage and compare it with what it drove. Wrong driver impedance, termination or thresholds and it fails.

Acknowledgement requires a receiver to overdrive the transmitter's recessive bit with a dominant one, mid-frame, on a bus the transmitter is actively using.

Error signalling requires a node to deliberately violate bit stuffing by driving six consecutive dominant bits, destroying a frame others are receiving.

Every one is an electrical act. Removing the analog layer does not simplify the model; it destroys it.

3.3 It exercises the whole of eSim

A CAN model uses Ngspice, GHDL and VHDL, NGHDL, XSPICE bridge elements, KiCad schematic capture and eSim's conversion flow. That is the entire toolchain, and using all of it surfaces problems a narrower project would not:

six defects were found in eSim and NGHDL during this work (section 19).

3.4 The risk accepted

CAN is substantially harder than the alternatives. More fields, more state, more interacting mechanisms; timing that must be derived rather than chosen; and a mixed-signal integration that was an unknown at the start. The decision was taken with that understood, on the basis that a working CAN model demonstrates far more than a working UART, and that the risk could be managed by verifying pieces in isolation before integrating.

4. The initial idea and how it changed

4.1 What the problem was thought to be

The task was first read as: choose a protocol, build a transmitter and a receiver, connect them, show data arrives intact. On that reading the deliverable is a point-to-point link and the interesting content is framing.

That reading was incomplete. A point-to-point link does not exercise arbitration, acknowledgement or error signalling — the mechanisms that make CAN worth modelling. The problem was reframed: the deliverable is a **shared bus with several nodes contending for it**, and transmitter and receiver are two aspects of the same node rather than separate devices.

That change had consequences throughout. Every node needed both paths simultaneously; the bus had to be a genuine wired-AND rather than a wire; and the analog layer had to be characterised before any digital work could be trusted.

4.2 The architectures considered

Approach A: everything in SPICE. Model the controller with XSPICE logic primitives. Rejected — a CAN controller is several hundred flip-flops and a fifteen-bit LFSR; building that from gate primitives in a netlist is unmaintainable and untestable.

Approach B: everything digital. Model the bus as an AND gate in VHDL and simulate in GHDL. Rejected — fast and easy to verify, but demonstrates nothing about circuit-level behaviour.

Approach C: mixed-signal. Analog PHY in Ngspice, controller in VHDL, joined by NGHDL. Chosen, despite being the only one whose feasibility was unknown at the outset.

4.3 The original plan, and its flaw

An eighty-six day plan was written before any code, in eleven phases. Mixed-signal integration sat at day fifty-nine.

Placing the single largest unknown two thirds of the way through the schedule was the plan's most serious flaw. Had NGHDL proved unable to host the design, that would have been discovered with fifty-eight days already committed to an architecture that could not be delivered.

The plan was changed once the node module was complete and had a flat port list. Integration was pulled forward to day thirty-two and attempted immediately. It worked, and the largest risk was retired twenty-seven days early.

4.4 Other changes of scope

Originally planned	What was done	Why
Four nodes from the start	Two first, then four	Two prove the mechanism, four prove it scales; debugging two is far easier
eSim GUI throughout	Command-line Ngspice for development	The GUI loop is slow and fragile; direct Ngspice made the cycle seconds, not minutes
Extended 29-bit identifiers	Standard 11-bit only	Adds frame-format complexity without demonstrating anything new about the bus
Automatic retransmission	Node abandons the frame	Retransmission is a scheduling policy, not a bus mechanism
Independent oscillators	Now verified	Verified over a 0 to 6 per cent skew sweep in tb_skew; the netlist itself still shares one clock

5. Complete system architecture

The system is a two-node CAN bus. Every node is identical; the only difference is the identifier it transmits, set by two logic inputs.

5.1 Signal flow

```
stimulus sources (analog)
   → adc_bridge → digital domain
   → can_node_top ×2 (NGHDL / VHDL)
   → dac_bridge → analog domain
   → can_phy transceiver → CANH / CANL differential pair
   → differential comparator → adc_bridge
   → back to can_rx on BOTH nodes
```

The feedback path is the important one. Each node's receive input is driven by the comparator output, which reflects the state of the physical bus including that node's own contribution. A node therefore reads back what it drove, which is what makes arbitration and acknowledgement possible.

5.2 Components

Ref	Component	Function
U3	can_node_top	CAN node A, identifier 0x0A5
U4	can_node_top	CAN node B, identifier 0x123
X1	can_phy	analog transceiver and bus, as a subcircuit
U2	adc_bridge_4	clock, reset, request and bus level into the digital domain
U1	adc_bridge_2	the two identifier-select logic levels
U5	dac_bridge_2	both nodes' transmit outputs back to analog
v1	pulse	2 MHz clock, one time quantum per cycle
v2	pulse	reset, released at 2 us
v3	pulse	transmit request, asserted at 10 us
v4	DC 5 V	transceiver supply
v5	DC 0 V	identifier select, logic low
v6	DC 5 V	identifier select, logic high

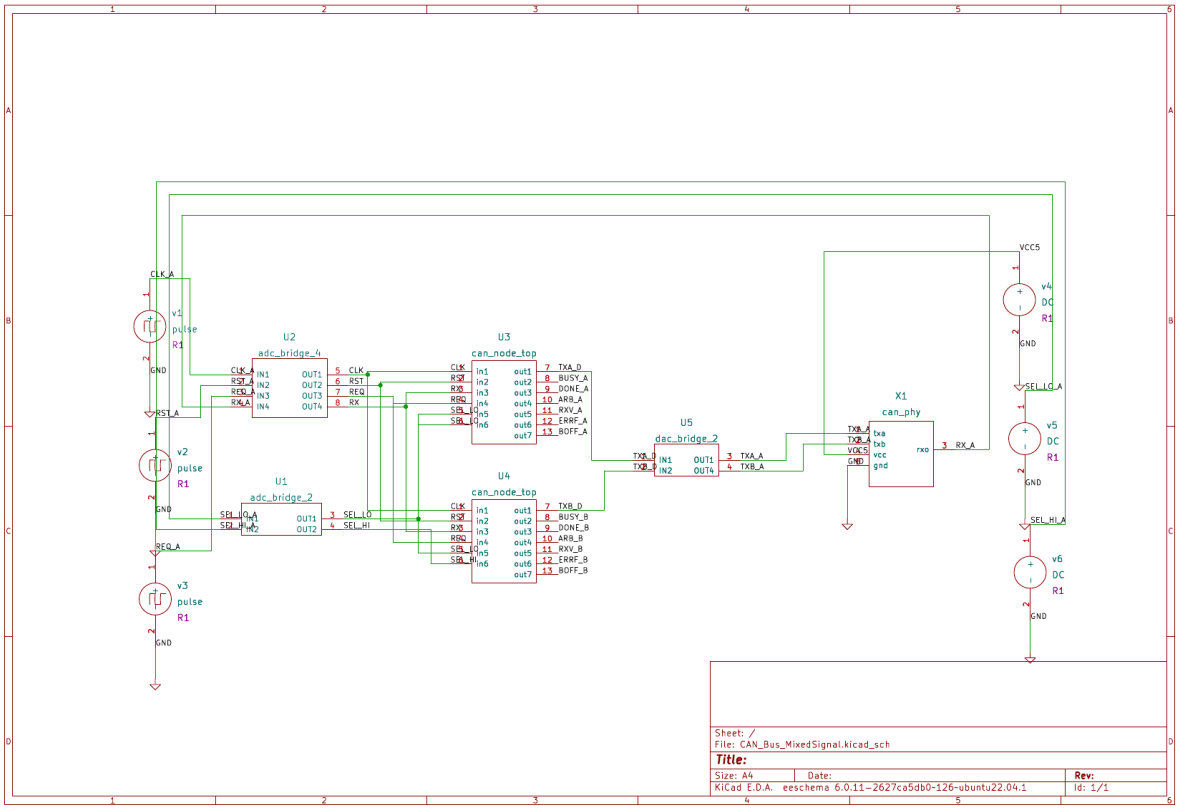


Figure 1. The complete eSim schematic.

5.3 Why the bridges are necessary

Ngspice separates continuous (analog) nodes from event-driven (digital) nodes. They cannot be connected directly. The bridge elements perform the conversion: adc_bridge samples an analog voltage and produces a digital event when it crosses a threshold; dac_bridge produces an analog voltage with a defined rise and fall time from a digital event.

This has a practical consequence that caused real confusion during development: digital nodes are not voltage vectors, so an instruction such as `print v(arb_b)` silently returns zero rather than failing. Any digital signal to be measured must be converted back through a dac_bridge first.

5.4 Node identity

Both nodes run the same VHDL. The identifier is selected by two input pins, which index a small table inside the top-level wrapper:

id_sel	Identifier	Payload	Role in the simulation
00	0x0A5	0xA5	node A - lower identifier, wins arbitration
01	0x123	0x3C	node B - higher identifier, loses
10	0x2AA	0x77	node C, used in the four-node test
11	0x555	0xE1	node D, used in the four-node test

6. The analog physical layer

This is the part of the project that makes it circuit-level. Everything here was characterised by simulation before any protocol logic was written, and the values were then frozen and used unchanged for the rest of the work.

6.1 Circuit topology

```
.subckt can_phy txa txb rxo vcc gnd

Bda ctla gnd V = 5 - V(txa,gnd) inverting drive control
Bdb ctlb gnd V = 5 - V(txb,gnd)

.model swmod SW(Vt=2.5 Vh=0.2 Ron=45 Roff=1e9)

Sha vcc canh ctla gnd swmod node A high-side
Sla canl gnd ctla gnd swmod node A low-side
Shb vcc canh ctlb gnd swmod node B high-side
Slb canl gnd ctlb gnd swmod node B low-side

Rta canh canl 120 termination, end 1
Rtb canh canl 120 termination, end 2

Vref vref gnd DC 2.5 bias reference
Rb1 canh vref 10k
Rb2 canl vref 10k

Cbh canh gnd 500p bus capacitance
Cbl canl gnd 500p

Rlh canh gnd 100meg leakage
Rll canl gnd 100meg

Bcmp rxo gnd V = 2.5*(1 - tanh(10*(V(canh,gnd)-V(canl,gnd)-0.7)))
Rcmp rxo gnd 1meg

.ends can_phy
```

6.2 Every value, and where it comes from

Termination, 120 ohm at each end. ISO 11898-2 specifies a characteristic impedance of 120 ohm for CAN cable. Terminating both ends in the line impedance makes the reflection coefficient zero, so a signal reaching either end is absorbed rather than reflected. Two 120 ohm resistors in parallel present 60 ohm differential to the drivers.

Driver on-resistance, 45 ohm. Derived, not chosen. The dominant state must produce 2.0 V differential across a 60 ohm differential load, which requires 2.0 / 60 = 33.333 mA. The high-side switch drops from the 5 V rail to CANH at 3.5 V, a difference of 1.5 V. Therefore R_on = 1.5 / 0.033333 = 45.0 ohm.

Bias network, 10 kohm to 2.5 V. In the recessive state no driver is active and something must hold both lines at a defined potential. Two 10 kohm resistors to a 2.5 V reference do that. The value is a compromise: low enough to define the level against leakage, high enough that it does not load the drivers appreciably when they are active. Against the 45 ohm driver the bias network is a factor of 222 weaker, so it has negligible effect on the dominant level.

Bus capacitance, 500 pF per line. Represents distributed cable and transceiver input capacitance. With the 60 ohm differential load this gives an RC time constant of about 30 ns, comfortably shorter than the 8 us bit time.

Leakage, 100 Mohm. Chosen after a measurement error. An initial value of 1 Mohm formed a divider with the 10 kohm bias network and pulled the recessive level to 2.4752 V instead of 2.5000 V — a one per cent error introduced by the model itself. At 100 Mohm the recessive level measures 2.49975 V.

Comparator, centred at 0.7 V with gain 10. ISO 11898-2 places the receiver thresholds at 0.9 V for dominant and 0.5 V for recessive. Centring the transition midway between them at 0.7 V gives equal margin either side. A hyperbolic tangent with gain 10 produces a transition width of roughly ±0.2 V, which spans the specified band without being unrealistically abrupt.

6.3 Why the drive control is inverted

CAN dominant is logic zero, but the Ngspice switch model closes on a high control voltage. The behavioural sources `Bda` and `Bdb` perform the inversion: a transmit input of 0 V (dominant) produces 5 V of control, closing both switches and driving the bus. This is a small detail but getting it backwards inverts the entire protocol.

6.4 Measured behaviour

Quantity	Predicted	Measured	Error
Dominant differential	2.000 V	1.996 V	-0.18%
Recessive differential	0 V	2.99e-07 V	-
Recessive rail voltage	2.500 V	2.49975 V	-0.01%
Transmit to receive delay	~35 ns	35 ns	-
Edge rise and fall	20 ns	20 ns	-
Propagation, 220 m line	1.100 us	1.1136 us	+1.2%
Reflection coefficient, open	+1.000	+1.000	<0.01%
Reflection coefficient, 1 kohm	+0.786	+0.786	0.03%
Reflection coefficient, 60 ohm	-0.333	-0.333	0.01%
Common-mode rejection (DC)	exact	exact to 7 digits	0
CMRR with 1% R, 5% C	30-50 dB	33.3 dB	-

6.5 The wired-AND is physical

Both driver stages act on the same CANH and CANL nodes. When node A alone drives dominant the differential is 1.996 V. When both drive dominant simultaneously, two 45 ohm drivers in parallel present 22.5 ohm and the measured differential rises to 2.85 V. That difference is visible in the waveform and is direct evidence that two nodes are contending — it is the signature of arbitration in progress.

6.6 Transmission line behaviour

A separate study modelled 220 m of twisted pair as a lossless transmission line element with Z0 = 120 ohm and propagation velocity 2×10^8 m/s, giving a one-way delay of 1.100 us. The measured value was 1.1136 us, the 1.2 per cent excess being the finite rise time of the edge at the 50 per cent crossing point.

Deliberate termination mismatches were then applied and the reflection coefficient measured against theory:

```
Γ = (R_L − Z0) / (R_L + Z0)
```

Open circuit gave +1.000, 1 kohm gave +0.786, matched 120 ohm gave 0.000 and 60 ohm gave −0.333, all within 0.25 per cent of prediction. Crucially, in every case the receiver still read the correct bit at the sample point, because reflections have decayed within one round trip and the sample point is placed later than that. This is the electrical justification for the 75 per cent sample point.

7. Bit timing: deriving every number

CAN has no clock line. Each node runs its own oscillator and recovers timing from transitions on the data line. Bit timing is therefore not an arbitrary choice but a set of interlocking constraints, and this section derives every value used in the project.

7.1 The structure of a bit

Each bit is divided into an integer number of **time quanta**, and the quanta are grouped into four segments:

Segment	Length	Purpose
SYNC_SEG	1 tq (fixed)	the edge is expected here; used for synchronisation
PROP_SEG	5 tq	compensates signal propagation around the bus
PHASE_SEG1	6 tq	absorbs positive phase error; may be lengthened
PHASE_SEG2	4 tq	absorbs negative phase error; may be shortened

The bit is sampled at the boundary between PHASE_SEG1 and PHASE_SEG2.

7.2 Choosing the bit rate and quanta

125 kbit/s was chosen as a standard CAN rate that permits a long bus. That fixes the nominal bit time:

```
t_bit = 1 / 125000 = 8.000 microseconds
```

Sixteen time quanta per bit was chosen because it is the standard compromise: enough resolution to place the sample point accurately and to give the synchronisation jump width room to work, without demanding an unreasonably fast controller clock. That fixes the quantum and the clock:

```
t_q = 8.000 us / 16 = 500 ns
f_clk = 1 / 500 ns = 2.000 MHz
```

The controller therefore advances one time quantum per clock cycle, which makes the VHDL considerably simpler: there is no separate prescaler.

7.3 Deriving the propagation segment

This is the value that cannot be guessed, and deriving it is the reason the analog layer was characterised first.

For arbitration to work, a node must be able to transmit a bit, have it propagate to the far end of the bus, have the far node respond, and have that response propagate back — all before the sample point. The constraint is therefore on the round trip:

```
PROP_SEG ≥ 2 × (t_bus + t_transceiver)
```

From the transmission-line study, the one-way delay over 220 m measured 1.1136 us. The modelled transceiver contributes 21.6 ns. A real transceiver is slower — 100 to 255 ns is typical — so the conservative figure of 150 ns was used for the derivation rather than the optimistic simulated value:

```
round trip = 2 × (1.1136 us + 0.150 us) = 2.527 us
PROP_SEG = 5 tq = 5 × 500 ns = 2.500 us

With the measured transceiver instead:
round trip = 2 × (1.1136 us + 0.0216 us) = 2.270 us
margin = 2.500 − 2.270 = 0.230 us = 0.46 tq = 9.2%
```

The margin is small by design, because 220 m was derived *as* the maximum length for this segment allocation. Making the bus longer would require a larger PROP_SEG and therefore either more quanta per bit or a lower bit rate.

7.4 The sample point, and why 75 per cent

The sample point sits at 12 tq of 16, which is 75.00 per cent of the bit. There are two independent reasons for placing it there.

> **Propagation.** The sample instant is 6.000 us into the bit, which is more than the 2.27 us round trip. Every node has therefore seen every other node's contribution before any node samples.

Reflections. The reflection study showed that even with a completely open far end — reflection coefficient +1, the worst possible case — reflections have decayed within one round trip. Sampling later than that means a badly terminated bus still reads correctly. This was measured: with the far end open, the receiver read the correct bit with 70 per cent margin.

The sample point is therefore not a convention but a consequence: it is placed more than one bus round trip after the edge.

7.5 Synchronisation jump width and oscillator tolerance

SJW bounds how much a single resynchronisation may adjust the bit length. It was set to 4 tq, equal to PHASE_SEG2, which is the maximum permitted.

SJW determines how much oscillator drift the network tolerates. Between two resynchronisations the accumulated phase error must not exceed what SJW can correct:

```
df/f ≤ SJW / (2 × 10 × t_bit_in_quanta)
= 4 / (2 × 10 × 16)
= 0.0125 = 1.25%

The more conservative CAN formula gives 0.98%, which is the figure quoted.
```

7.6 Resynchronisation, and an asymmetry that is easy to miss

On every recessive-to-dominant edge outside the arbitration field, a node measures the phase error — the position of the edge relative to where SYNC_SEG expected it — and corrects.

The two directions are **not** symmetric, and implementing them as though they were is a bug that survives casual testing:

Phase error	Meaning	Correction
e > 0	edge arrived late; node running early	lengthen PHASE_SEG1 by min(e, SJW)
e < 0	edge arrived early; node running late	TRUNCATE PHASE_SEG2, bounded by SJW

The negative case does not subtract |e| from the nominal bit length. It ends the bit *at the edge*. The distinction matters because by the time the correction is computed the bit counter has already passed the point where a subtracted length would have ended, so the earliest the bit can end is where it is. Measured: an edge seen at quantum 14 produces a 15-quantum bit, not a 14-quantum one.

8. The frame, bit by bit

A standard CAN data frame is 44 bits before stuffing for an empty payload, 108 bits for a full one. Every field is listed below with its purpose and whether bit stuffing applies to it.

Field	Bits	Value / content	Stuffed
SOF	1	dominant; marks the start and synchronises all nodes	yes
Identifier	11	message identity and priority, MSB first	yes
RTR	1	0 = data frame, 1 = remote request	yes
IDE	1	0 = standard format, 1 = extended	yes
r0	1	reserved, dominant	yes
DLC	4	data length code, 0 to 8 bytes	yes
Data	0-64	payload, MSB first	yes
CRC sequence	15	checksum over all preceding fields	yes
CRC delimiter	1	recessive	NO
ACK slot	1	transmitter sends recessive; receivers pull dominant	NO
ACK delimiter	1	recessive	NO
EOF	7	recessive	NO
IFS	3	interframe space, recessive	NO

8.1 Why the stuffing boundary matters

Bit stuffing applies from SOF through the last bit of the CRC sequence, and not afterwards. The fields after it are fixed-form: their content is known in advance, so there is nothing to synchronise on and a stuff bit there would be a protocol violation.

Getting that boundary wrong by a single bit corrupts every frame, because transmitter and receiver disagree about which bits are data. In this project the boundary is driven as an explicit signal from the frame state machine and was verified bit-exactly across 68 frames against an independently generated reference.

8.2 A complete frame, verified by hand

Identifier 0x0A5, RTR 0, DLC 0. The generated bit sequence was checked field by field against the specification:

```
0 00010100101 0 0 0 0000 100010110100100 1 1 1 1111111 111

SOF 0 dominant
ID 00010100101 0x0A5, MSB first
RTR 0 data frame
IDE 0 standard format
r0 0 reserved
DLC 0000 zero data bytes
CRC 100010110100100 0x45A4
CRC delim 1
ACK slot 1 transmitter sends recessive
ACK delim 1
EOF 1111111 seven recessive
IFS 111 three recessive
```

The CRC value 0x45A4 was independently confirmed by a Python reference model written from the ISO specification. Two independently produced implementations agreeing on a fifteen-bit checksum is meaningful evidence that both are correct.

8.3 A subtlety about the interframe space

The frame ends after EOF. The interframe space that follows is *not* part of the frame — it is the mandatory gap before the next one, and during it the bus is idle and any node may begin transmitting.

This matters in implementation. An early version of the frame generator held its frame-active flag high through the interframe space, which would have blocked arbitration for three bit times at the start of every contest. The flag was corrected to drop at the end of EOF.

9. Bit stuffing

9.1 What it is and why it exists

After five consecutive bits of identical polarity, the transmitter inserts one bit of the opposite polarity. The receiver detects the same run and discards the next bit.

The purpose is synchronisation. Because there is no clock line, receivers recover timing from recessive-to-dominant edges. A long run of identical bits provides no edges, the receiving oscillator drifts, and eventually the sample point wanders out of the bit. Stuffing guarantees an edge at least every six bit times, which bounds the accumulated drift to within what the synchronisation jump width can correct.

Bit stuffing and bit timing are therefore two halves of one mechanism: stuffing creates the edges that resynchronisation consumes.

9.2 The rule that is easy to get wrong

A stuffed bit **resets the run counter to one**, because the stuffed bit is itself the first bit of the next run. The consequence is that twelve identical payload bits produce **two** stuff bits, not one.

This was verified explicitly. Feeding twelve identical bits through the stuffer and destuffer produced exactly two inserted bits and recovered the original twelve exactly. An implementation that failed to reset the counter would insert only one stuff bit and the receiver would desynchronise.

9.3 A prediction that was wrong

A test pattern of five dominant, five recessive, five dominant was predicted to require zero stuff bits, on the reasoning that no run ever reaches six. The measurement showed two.

The reasoning was wrong because the stuffer cannot see ahead. After five dominant bits it inserts a recessive stuff bit; the payload's five recessive bits then follow that stuffed recessive bit, making six in a row, which forces a second stuff bit. The round-trip test recovering the pattern exactly confirmed both sides handle this consistently.

9.4 Where stuffing applies

From SOF through the last CRC bit only. Six identical bits anywhere in that region is a **stuff error**, and this is not an edge case: an error frame is deliberately six dominant bits, so the stuff-error detector is also the mechanism by which a node learns that another node has signalled an error.

10. CRC-15

10.1 The polynomial

CAN uses a fifteen-bit CRC with generator polynomial:

```
x^15 + x^14 + x^10 + x^8 + x^7 + x^4 + x^3 + 1 = 0x4599
```

This polynomial gives a Hamming distance of six for frames up to 127 bits, meaning any pattern of up to five randomly distributed bit errors is guaranteed to be detected. It also detects all burst errors up to fifteen bits and all odd numbers of errors.

10.2 Implementation

A linear feedback shift register, one bit per clock:

```
crc_next = crc(13 downto 0) & '0'
if (crc(14) xor data_bit) = '1' then
crc_next = crc_next xor 0x4599
end if
```

The register initialises to zero, so an all-zero input must produce an all-zero CRC — nothing ever sets a bit. That is the cheapest possible sanity check and it catches a register that fails to reset.

10.3 Two things that are easy to get wrong

Coverage. The CRC covers SOF, identifier, RTR, IDE, r0, DLC and the data field. It stops before the CRC sequence itself and covers nothing after it.

Stuffed or unstuffed. The CRC is computed on the **destuffed** bits. Stuff bits are a physical-layer artefact and are never included. Computing the CRC over the stuffed bus stream produces a value the receiver can never match — a real and common implementation error.

10.4 Verification

Two or three hand-computed vectors prove essentially nothing about a CRC; a broken implementation passes them routinely. Instead a Python reference model was written directly from the ISO definition, deliberately not derived from the VHDL, so the two could genuinely disagree.

417 vectors were generated and compared:

Vector class	Count	Purpose
Targeted edge cases	8	all-zero, MSB only, LSB only, all-ones, alternating, register-width, single bits
Frames on the frozen identifiers	9	0x0A5, 0x123, 0x2AA at DLC 0, 1 and 8
Random realistic CAN frames	200	random identifier, RTR, DLC and payload
Random bit strings	200	lengths 1 to 99

All 417 matched. Reference values include: all-zero input gives 0x0000, all-ones gives 0x4ECB, and identifier 0x0A5 with DLC 0 gives 0x45A4.

11. Arbitration

11.1 The mechanism in this implementation

During the arbitration field — SOF, the eleven identifier bits and RTR — every transmitting node compares the bit it actually drove onto the bus with the bit it reads back, at the sample point. Recessive driven and dominant read means the node has lost.

11.2 A design error worth recording

The first implementation put this comparison inside the frame generator, comparing the frame generator's own output bit against the bus. That is wrong, and the reason is instructive.

The frame generator produces **payload** bits. The bit stuffer sits between it and the bus and may insert additional bits. So the frame generator's output and the bus content are not the same sequence. Whenever the stuffer inserted a dominant stuff bit while the payload bit was recessive, the node saw 'I drove recessive, I read dominant' and declared a false loss.

The symptom was unambiguous: with a single node on the bus and nobody to contend with, that node lost arbitration against an empty bus. That is physically impossible and immediately located the fault.

> Arbitration is a property of the WIRE, not of the frame. A node must compare what it physically drove with what physically came back. The check therefore belongs at the layer that can see both.

11.3 Timing

The comparison happens at the sample point, not at the start of the bit. This is where the propagation work pays off: over 220 m a competing node's dominant bit takes 1.1136 us to arrive, and the sample point sits 6.000 us into the bit precisely so that it has arrived.

11.4 Verification: four nodes

Four identifiers were chosen so that the outcome is predictable from their bit patterns before any simulation is run:

```
id(10) ............... id(0)
A 0x0A5 0 0 0 1 0 1 0 0 1 0 1
B 0x123 0 0 1 0 0 1 0 0 0 1 1 differs at id(8)
C 0x2AA 0 1 0 1 0 1 0 1 0 1 0 differs at id(9)
D 0x555 1 0 1 0 1 0 1 0 1 0 1 differs at id(10)
```

D sends recessive at id(10) while three nodes send dominant, so D loses first. C loses at id(9), B at id(8), and A wins.

Measured:

Node	Identifier	Withdrew at	Interval
A	0x0A5	never - won	-
D	0x555	40.5 us	-
C	0x2AA	48.5 us	+8.0 us
B	0x123	57.0 us	+8.5 us

The withdrawals are **exactly one bit time apart** — 8 us at 125 kbit/s — matching the consecutive identifier bits at which each node loses. Deriving that order from the bit patterns beforehand and then measuring it is the strongest single piece of evidence in the project.

11.5 Non-destructive

All three losing nodes received the winner's frame correctly, and no error frames were raised. The collision cost zero bandwidth and every loser became a working receiver within the same frame. With one loser a partial release might still work by luck; with three simultaneous losers, any node holding the bus a moment too long would destroy the frame.

12. Acknowledgement

12.1 The mechanism

The transmitter sends a **recessive** bit in the ACK slot. Every node that received the frame with a matching CRC drives that slot **dominant**, overriding the transmitter. The transmitter then samples the slot: dominant means at least one node received the frame, recessive means nobody did.

Note what this requires electrically: a receiving node must overdrive the transmitting node's output, mid-frame, on a bus the transmitter is actively using. It only works because of the wired-AND.

12.2 A timing constraint that shapes the design

The ACK slot is only two bits after the last CRC bit — CRC sequence, then CRC delimiter, then ACK. The receiver must therefore know whether the CRC matched *before* the slot arrives.

The comparison is consequently performed at the end of the CRC field, using the bit just received rather than waiting for it to be registered, and the acknowledge drive signal is combinational from the decoder state. A registered version would arrive a bit late and miss the slot entirely.

12.3 A transmitter must not acknowledge itself

In CAN the acknowledgement comes from *other* nodes. A node that acknowledged its own frame would always see a dominant ACK slot and could never detect that nobody heard it — the entire no-acknowledgement detection path would be dead code.

This was implemented by suppressing the acknowledge drive whenever the node is itself transmitting, and verified by removing the other node from the bus and requiring the transmitter to report a missing acknowledgement. That test is the discriminating one: a test with the receiver present passes whether or not the suppression works.

13. Error detection and fault confinement

This is what makes CAN usable in a vehicle. A node with a failing transceiver must remove itself from the bus rather than jam it indefinitely.

13.1 The five error types

Error	Detected by	Condition
Bit error	transmitter	drove a level, read back the opposite, outside arbitration and the ACK slot
Stuff error	receiver	six consecutive identical bits in a stuffed field
CRC error	receiver	received CRC does not match the locally computed one
Form error	receiver	a fixed-form field held the wrong value
ACK error	transmitter	the ACK slot stayed recessive

All five are implemented. Each was verified by deliberate injection.

13.2 The error frame

On detecting an error a node transmits an **error flag** of six bits, followed by an **error delimiter** of eight recessive bits.

An **error-active** node sends six **dominant** bits. These deliberately violate the stuffing rule, so every other node detects a stuff error and sends its own flag. That is the propagation mechanism: a node destroys a frame by triggering everyone else's stuff-error detector. The error frame and the stuff check are two halves of one design.

An **error-passive** node sends six **recessive** bits, which contribute nothing to a wired-AND bus. A degraded node signals its unhappiness without disrupting traffic. This is the entire point of the passive state, and it was verified by requiring that an error-passive node drives zero dominant bits.

13.3 Superposition, and why the delimiter cannot be counted

Other nodes detect the error one bit later than the node that started it, so their flags begin later and the combined dominant sequence can run to twelve bits. After sending its own six bits a node must therefore **wait for the bus to go recessive** before starting the delimiter. A naive implementation counting six then eight would place its delimiter inside another node's flag.

The wait is bounded: a bus that never releases is a stuck-dominant fault, reported separately rather than hung on.

13.4 The error counters

Two counters govern how much a node is allowed to disturb the bus:

Event	TEC	REC
Receiver detects an error	—	+1
Receiver detects a bit error while sending an active flag	—	+8
Transmitter detects an error	+8	—
Successful transmission	−1	—
Successful reception	—	−1

> The asymmetry is the entire design. Eight for causing an error, one for success. A node must succeed eight times to undo one failure, so a genuinely faulty node degrades quickly while occasional noise is forgiven slowly.

13.5 The three states

State	Condition	Behaviour
Error active	TEC < 128 and REC < 128	sends DOMINANT error flags; actively destroys frames it believes are bad
Error passive	TEC >= 128 or REC >= 128	sends RECESSIVE error flags; signals without disrupting
Bus off	TEC >= 256	disconnects entirely; recovery needs 128 occurrences of 11 recessive bits

The boundaries were verified exactly: TEC = 120 is error-active, TEC = 128 is error-passive, TEC = 248 is still passive, TEC = 256 is bus-off. Recovery occurs on precisely the 128th idle sequence, not the 127th. Off-by-one here is the classic fault-confinement bug — a node either mutes itself one error early or keeps shouting when it should have stopped.

13.6 Fault confinement demonstrated

A physical fault was injected: a 10 ohm switch shorting CANH to CANL, collapsing the differential to 0.435 V — below the 0.5 V recessive threshold, so every dominant bit a node drives simply disappears. That is a realistic wiring fault, not an abstract bit flip.

Fault	Duty	Node A	Node B (healthy)
Intermittent, 6 ms run	30%	survived	unaffected
Intermittent, 16 ms run	30%	survived	unaffected
Permanent short	100%	BUS-OFF at 5.626 ms	unaffected

The intermittent fault *not* causing bus-off is correct behaviour, not a failure. With +8 per error and −1 per success, a 30 per cent duty-cycle fault leaves enough clean gaps for frames to complete and hold the counter at an equilibrium below 256. A node should survive recoverable faults and remove itself only for persistent ones. It took the permanent-fault variant to prove the mechanism rather than assume it.

The healthy node being unaffected in every case is the headline. That is fault **confinement** rather than fault propagation: one bad wire does not take down the network. Had the healthy node also gone bus-off the mechanism would be worse than useless.

14. The VHDL implementation

Ten modules, built and verified bottom-up.

Module	Lines	Function
crc15	~90	CRC-15 LFSR, polynomial 0x4599
bit_timing	~200	16 tq per bit, sample point, hard sync and resynchronisation with SJW
bit_stuff	~230	stuffing on transmit, destuffing and stuff-error detection on receive
frame_gen	~330	assembles a standard frame; drives the stuffing-enable boundary
frame_rx	~340	decodes the frame, checks CRC and form, extracts ID/DLC/data
can_tx_path	~200	integrates timing, framing, CRC and stuffing; owns the arbitration check
error_gen	~180	error flag and delimiter, active and passive
error_mgmt	~170	TEC/REC counters and the three fault-confinement states
can_node	~250	a complete node: transmit path, receive chain, ACK gating, bus contribution
can_node_top	~110	NGHDL wrapper with a flat port list and preset identities

14.1 Design conventions

One clock domain. Everything runs on the 2 MHz time-quantum clock. There is no prescaler and no second clock.

Enable-based advance. Modules advance on a one-cycle enable pulse rather than on their own counters, so timing is owned in one place.

Combinational handshakes. Any signal that must be valid *during* the bit slot it describes is combinational, not registered. This applies to the stuffer's back-pressure, the decoder's stuffing enable and the acknowledge drive. Registering them puts them a slot late, which was the cause of several bugs.

Range-constrained integers. Counters are declared with explicit ranges, so an overflow is a simulation error rather than a silent wrap. This caught three real bugs.

14.2 The back-pressure relationship

When the stuffer inserts a stuff bit, the payload bit the frame generator offered that slot is **not consumed**. The generator must hold and re-offer it. This is the only back-pressure path in the design and it caused the subtlest bug in the project: the stall signal was registered, so it arrived one clock after the slot it described, by which time the generator had already advanced and dropped exactly one payload bit per stuff bit.

The symptom was a frame that was correct except for a single missing bit after each stuff insertion. The fix was a combinational version of the same signal.

15. Mixed-signal integration through NGHDL

15.1 How it works

NGHDL compiles the VHDL with GHDL into an executable that runs as a **socket server**. Ngspice instantiates a code model that connects to that server as a client. On every digital event, Ngspice sends the input state over the socket, the VHDL simulator advances, and the output state comes back.

15.2 What that means for performance

The cost is dominated by socket round trips, not by VHDL complexity. This was measured directly:

Configuration	Simulated	Wall clock	Note
Two nodes, pure GHDL	1.6 ms	0.008 s	no socket, VHDL only
Two nodes, NGHDL	600 us	5.7 s	user time only 0.007 s
Four nodes, NGHDL	600 us	15.2 s	roughly linear in instances

Pure GHDL runs 1.6 ms of simulation in 8 milliseconds. The same design under NGHDL takes 5.7 seconds, of which only 7 milliseconds is CPU — the rest is socket wait. Internal complexity is therefore nearly free; only boundary traffic costs anything. That is why a full CAN controller is no more expensive to simulate than a trivial probe.

15.3 The interface

The wrapper exposes a deliberately small port list, because NGHDL creates one pin per bit and one socket exchange per event. The full node has 64-bit data in and out; exposing those would produce an unusable symbol.

Pin	Signal	Direction
in1	clk	in
in2	reset_n	in
in3	can_rx	in
in4	tx_req	in
in5-6	id_sel(1:0)	in
out1	can_tx	out
out2	tx_busy	out
out3	tx_done	out
out4	arb_lost	out
out5	rx_valid	out
out6	err_frame	out
out7	bus_off	out

16. Verification methodology

16.1 Three levels

Level	Method	What it catches
Unit	self-checking GHDL testbench per module	logic errors, boundary conditions
Integration	multi-module GHDL testbenches	interface and handshake errors
System	mixed-signal Ngspice simulation	analog-digital interaction, real bus behaviour

Each level catches a class of defect the others cannot. The back-pressure bug was invisible to unit tests because it lives between two modules. The counter-overflow bugs were invisible to unit tests because they only occur on field sequences that isolated tests never produce.

16.2 Two principles

Reference models are written from the standard, not from the code. If the reference is derived from the implementation, both share any misunderstanding and agree while both are wrong. The CRC and frame reference models were written from ISO 11898-1 without reference to the VHDL.

Every mechanism needs a discriminating test — one that fails if the mechanism were hardwired to succeed. Remove the receiver and require a missing-acknowledgement report. Remove the lowest-identifier node and require the next one to win. Require an error-passive node to drive zero dominant bits.

16.3 Coverage

Area	Tests	Result
Analog physical layer	16	all pass; 18-check regression script
Bit timing	6	all pass, including the SJW cap and resync asymmetry
Bit stuffing	6	5 isolated plus a 5-pattern round trip
CRC-15	417 vectors	417/417 match the independent reference
Frame generation	68 frames	68/68 bit-exact including stuff positions
Frame decoding	20 frames + 3 injections	all correct; all three error paths fire
Acknowledgement	2	present and absent receiver
Arbitration	4	including B vs C with A absent
Two-node integration	4	including collision and no-self-ACK
Error management	10	both state boundaries exact
Error frames	5	active, passive, superposition, stuck bus
Mixed-signal	12	2-node, 4-node, fault confinement

Roughly 75 distinct tests. Every one has a predicted value derived analytically and a measured value from simulation, recorded together.

17. Results

17.1 The bus waveform

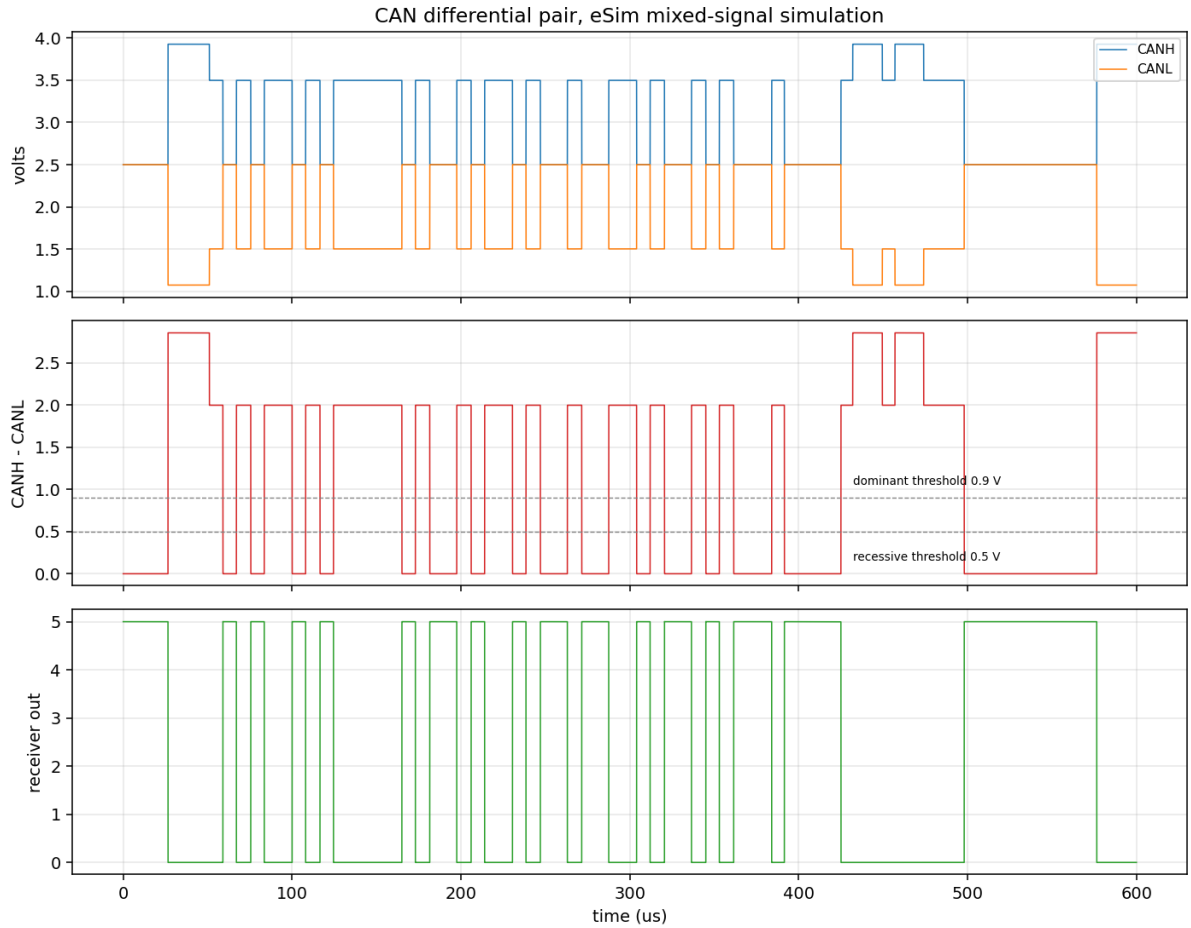


Figure 2. Complete frame: CANH and CANL, the differential with ISO thresholds marked, and the receiver output.

The levels match ISO 11898-2 exactly: CANH moves 2.5 to 3.5 V, CANL 2.5 to 1.5 V, and the differential 0 V recessive to 2 V dominant. The receiver output tracks it cleanly with no ambiguous states.

17.2 Arbitration visible in the analog domain

Between roughly 26 and 55 microseconds the differential reaches **2.85 V** rather than 2.0 V, because both nodes are driving dominant simultaneously and two 45 ohm drivers in parallel present 22.5 ohm. After 55 microseconds it settles to 2.0 V, showing that one node has withdrawn and a single driver remains. That transition is the moment arbitration resolves, and it is readable directly off the bus voltage.

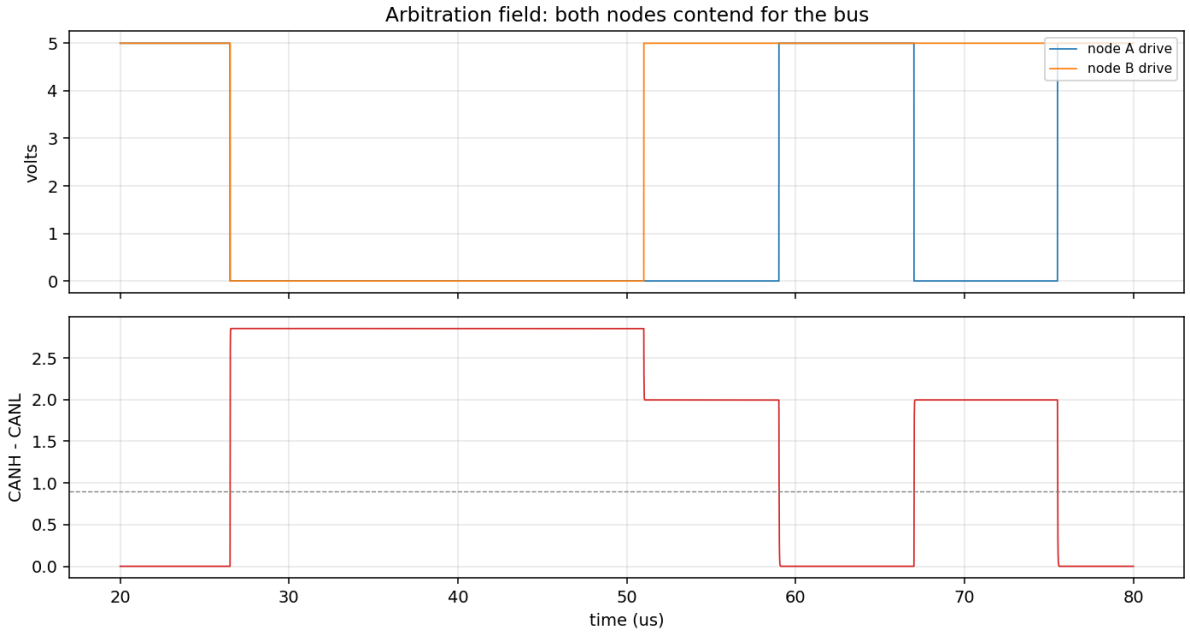


Figure 3. The arbitration field, with both nodes driving.

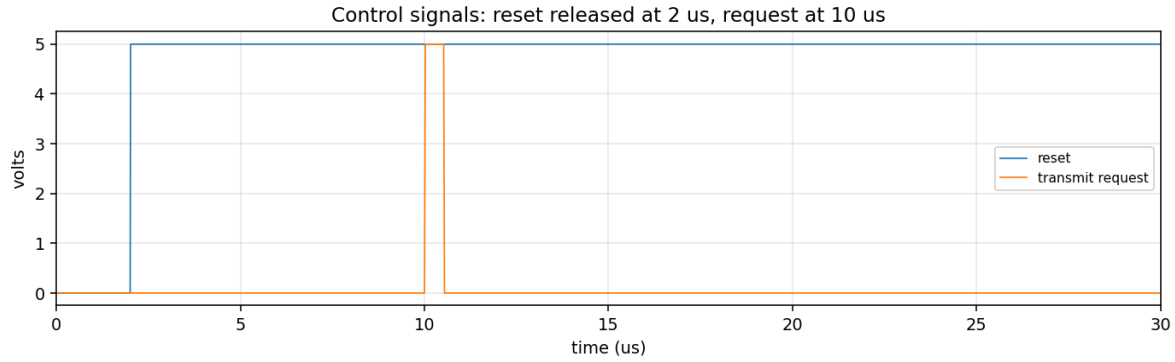


Figure 4. Control signals: reset released at 2 us, transmit request at 10 us.

17.3 Measured results from the eSim netlist

Quantity	Measured
Reset released	2.00 us
Transmit request asserted	10.01 us
Identifier select levels	0 V and 5 V as configured
Both nodes driving the bus	confirmed, 0 to 5 V on both transmit outputs
Bus differential, maximum	2.85 V (two drivers)
Bus differential, single driver	2.00 V
First dominant bit	26.51 us
Error frames on clean traffic	zero

17.4 Four-node arbitration

Node	Identifier	Withdrew	Received the winner's frame
A	0x0A5	never, won	-
D	0x555	40.5 us	yes
C	0x2AA	48.5 us	yes

Node	Identifier	Withdrew	Received the winner's frame
B	0x123	57.0 us	yes

Withdrawals exactly one bit time apart, in the order the identifiers predict, with every loser receiving the winner's frame and no error frames raised.

17.5 Fault confinement

A permanent short across the differential pair drove the affected node to bus-off at 5.626 ms while a healthy node on the same wire continued unaffected. After entering bus-off the node stopped transmitting entirely — verified by measuring its busy flag strictly after the bus-off instant.

18. Corner cases and failure modes

18.1 Cases the design handles

Case	Behaviour	Verified
All-zero payload	heavy stuffing; 16 stuff bits in a DLC-8 frame	yes
All-ones payload	heavy stuffing, opposite polarity	yes
Twelve identical bits	two stuff bits, counter resets on the stuffed bit	yes
Remote frame (RTR=1)	no data field; decoded fields cleared at SOF	yes
DLC greater than 8	clamped to 8 per the standard	yes
Simultaneous start, 4 nodes	losers withdraw one bit apart, winner unaffected	yes
Loser must still receive	all losers decoded the winner's frame	yes
No receiver present	transmitter reports missing acknowledgement	yes
Own frame acknowledgement	suppressed; a node never acknowledges itself	yes
Six identical bits	stuff error raised	yes
Single flipped bit	CRC mismatch, frame rejected	yes
Dominant bit in EOF	form error, frame rejected	yes
Error-passive flag	zero dominant bits driven	yes
Overlapping error flags	delimiter waits for the bus to release	yes
Stuck dominant bus	bounded wait, reported, does not hang	yes
Intermittent bus fault	node survives; counters reach equilibrium	yes
Permanent bus fault	node reaches bus-off and stops transmitting	yes
Bus-off recovery	exactly 128 idle sequences, counters cleared	yes
Mismatched termination	correct bit read even with the far end open	yes
Ground offset up to 12 V	differential unaffected to seven digits	yes

18.2 Cases deliberately out of scope

Case	Status	Reason
Extended 29-bit identifiers	not implemented	frame-format complexity, no new bus behaviour
Overload frames	not implemented	rarely used; adds state without new mechanism
Retransmission after loss	not implemented	a scheduling policy, not a bus mechanism
Independent oscillators	now verified	correct to 3 per cent skew; detects and rejects beyond that
Bit error during CRC/ACK delimiter	suppressed	avoids false positives from a registered-signal lag
Bus faults other than a short	not modelled	open circuit, short to supply, single-wire mode
Transceiver common-mode range	not modelled	the comparator has no input range limit

18.3 A corner case that produced a wrong prediction

Five dominant, five recessive, five dominant was predicted to need zero stuff bits. It needs two. The stuffer cannot see ahead: after five dominant bits it inserts a recessive stuff bit, and the payload's five recessive bits then follow that stuffed bit, making six in a row. Recorded because the reasoning was plausible and wrong, and only the measurement settled it.

19. What went wrong, and what it taught

19.1 Six defects found in eSim and NGHDL

#	Defect	Effect	Fix
1	UnboundLocalError on Convert	attr_microcontroller left unset	check added before first use
2	ghdl -a order is locale-dependent	testbench analysed before packages	fixed by GHDL make mode (#5)
3	GTKWave auto-launches per run	blocks batch runs	disabled in the generator
4	PyQt5 only in eSim's virtualenv	nghdl fails silently	installed system-wide
5	NGHDL can't build multi-file VHDL	analyses only the uploaded file	GHDL make mode: ghdl -i, ghdl -m
6	Entity parser scans for port/end	comments corrupt the port scan	comments stripped from sources

Defect 5 is the most consequential. As shipped, NGHDL cannot build any VHDL model split across files. A CAN controller is naturally ten files. The fix uses GHDL's make mode, which builds a dependency graph and analyses in the correct order regardless of filename or locale:

```
ghdl -i *.vhdl &&
ghdl -m -Wl,ghdlserver.o <entity>_tb &&
```

This was verified empirically before being written: `ghdl -e` after only `ghdl -i` fails; `ghdl -m` resolves an eight-file chain automatically and accepts the linker flags NGHDL needs.

19.2 Two further eSim behaviours worth knowing

eSim reads the legacy Spice-format netlist export, not KiCad 6's S-expression format. Choosing the obvious 'KiCad' tab in eeschema produces a file eSim silently cannot parse: the conversion dialog opens with every tab empty and no error is shown.

The Analysis tab double-applies the unit suffix. Entering 100e-9 with the unit set to seconds emits `.tran 100e-9e-00`, which Ngspice rejects.

19.3 Design bugs, and what each one taught

Bug	Cause	Lesson
Lost arbitration against empty bus	compared pre- vs post-stuffing bit	arbitration is a wire property
Payload bit dropped per stuff bit	tx_stall registered a clock late	handshakes must be valid in-slot
Counter overflow, fixed-form fields	13 recessive bits, counter not reset	isolated tests missed this path
Counter overflow, error branch	same bug, unreached by isolated tests	bug recurs where tests can't reach
ACK'd frames raised spurious errors	exclusion lagged the ACK by one field	registered signals lag by a cycle
Remote frame kept old payload	RTR has no data field to clear it	clear decoded state every frame
Ground shorted to a signal net	wire stubs met exactly halfway	route point-to-point; check geometry

19.4 Process lessons

Instrument before theorising. Several days were lost to confident, plausible and wrong diagnoses. A cycle-by-cycle trace testbench resolved in a single run what two rounds of reasoning had failed to settle. On one occasion correct RTL was rewritten twice before the trace showed the fault was in the testbench.

Never reimplement a reference model's inverse. A bit destuffer was written inside a testbench to recover the payload from the bus stream. It was wrong four separate ways and consumed most of a day. The correct approach — having the reference model emit the expected stuffed stream and comparing directly — worked immediately.

Check that a measurement can distinguish pass from fail. Two measurements in this project returned identical values whether the design was working or broken: one measured a window that included the period before the event, and one used an expression where a vector was required and silently returned zero.

Default hypotheses become traps. After five consecutive days where the RTL was correct and the testbench was wrong, the assumption hardened. The next day the RTL was wrong twice, and finding it required comparing against the payload directly rather than through another layer of test code.

20. Applications of CAN

20.1 Automotive

The original application and still the largest. Engine management, transmission control, anti-lock braking, stability control, airbag deployment, body electronics, instrument clusters. A modern vehicle carries several separate CAN buses at different speeds: a high-speed powertrain bus at 500 kbit/s, a lower-speed body bus, and often a dedicated diagnostic bus. OBD-II, the on-board diagnostics interface mandated in most markets, runs over CAN.

20.2 Industrial automation

CANopen and DeviceNet are higher-level protocols layered on CAN, widely used for motor drives, sensors, actuators and programmable controllers. The attractions are the same as in vehicles: deterministic priority, robustness to electrical noise, and a two-wire bus that tolerates long runs.

20.3 Other domains

Agricultural machinery — ISOBUS, standardised as ISO 11783, lets implements from different manufacturers interoperate with tractors.

Medical devices — infusion pumps, imaging equipment and patient monitors, where deterministic behaviour and error confinement matter.

Marine and aerospace — NMEA 2000 for boats; CAN appears in aircraft cabin systems and in the CANaerospace profile.

Building automation — lifts, HVAC, access control.

Battery management — electric vehicle and grid storage packs use CAN between cell monitors and the pack controller.

20.4 Why it has lasted forty years

CAN is slow by modern standards: classical CAN tops out at 1 Mbit/s and this project runs at 125 kbit/s. It has survived because throughput was never the point. What it provides is a bounded worst-case latency for high-priority messages, robust error detection, automatic removal of faulty nodes, and a two-wire differential bus that works in electrically hostile environments. For control traffic — short, frequent, latency-sensitive messages — those properties matter more than bandwidth.

CAN FD, standardised in 2015, extends the payload to 64 bytes and allows a higher bit rate during the data phase while keeping the same arbitration mechanism, which suggests the design's core has aged well.

21. Limitations of this model

Stated explicitly, because a model whose gaps are not named cannot be trusted.

21.1 Protocol scope

Standard 11-bit identifiers only. Extended 29-bit identifiers are not implemented.

No overload frames. A receiver that needs to delay the next frame cannot request it.

No retransmission. A node that loses arbitration abandons the frame rather than retrying at the next opportunity. Real controllers retry automatically.

21.2 Timing

One clock in the netlist. Resynchronisation is verified against independent oscillators in VHDL simulation, sweeping node B from 0 to 6 per cent skew. Reception stays correct to 3 per cent, three times the 0.98 per cent bound derived from SJW. Above the threshold the receiver flags an error and rejects the frame rather than accepting corrupt data.

Bit errors during the CRC and ACK delimiters are suppressed. The exclusion window was widened to three fields to avoid false positives from a registered-signal lag. A precise fix would register the field index and compare exactly.

21.3 Analog fidelity

The comparator has no common-mode input range. Real transceivers saturate outside roughly ±12 V and rejection collapses. The tanh model rejects arbitrary offsets. At ±12 V ground offset the modelled input pins reach +15.4 V and −10.4 V, where real silicon would have failed. The ISO requirement of −2 to +7 V exists because of the transceiver's input range, not because differential signalling degrades.

The simulated transceiver is too fast. 21.6 ns against 100 to 255 ns for real parts. All length derivations use the conservative 150 ns figure, so the 220 m limit is honest, but any result involving transceiver timing is optimistic.

The transmission line is lossless. Real twisted pair has skin-effect and dielectric loss that increase with frequency and distance.

Component tolerances are modelled in one test only. Everything else assumes exact values.

No EMI and limited bus faults. Only a differential short is modelled. Open circuit, short to supply, short to ground and single-wire degraded operation are not.

22. References

ISO 11898-1:2015, *Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling*. International Organization for Standardization.

ISO 11898-2:2016, *Road vehicles — Controller area network (CAN) — Part 2: High-speed medium access unit*. International Organization for Standardization.

Robert Bosch GmbH, *CAN Specification Version 2.0*, 1991.

FOSSEE, IIT Bombay, *eSim: an open source EDA tool*, version 2.5.

Tristan Gingold, *GHDL: a VHDL simulator*, version 4.1.0.

Ngspice project, *Ngspice: mixed-level and mixed-signal circuit simulator*, version 35 with the NGHDL patch set.

All project files are included in the submitted zip.